

Bash & Proxmox Befehls-Referenz

Konfiguration finden, Services diagnostizieren, Doku pflegen und mit Git auf GitHub hochladen

Diese Referenz sammelt die Befehle, die im Homelab-Projekt immer wieder gebraucht werden: herausfinden, wie ein Service läuft, seine Konfiguration finden, und die Ergebnisse sauber ins GitHub-Repository bringen.

1. Wo führe ich was aus?

Der häufigste Stolperstein: der falsche Ort für einen Befehl.

Ort	Wofür	Erkennbar an
Proxmox-Host-Shell	pct-Befehle, Container von außen steuern	Prompt: root@pve:~#
Innerhalb eines LXC-Containers	Service-eigene Befehle (systemctl, docker, php)	Prompt: root@<containername>:~#
Git Bash (Windows-PC)	git-Befehle, scp zum Runterladen von Dateien	Pfad beginnt mit /c/...
PowerShell (Windows)	Nicht für git/scp geeignet ohne Zusatzinstallation	PS C:\...

2. Container von außen steuern (pct)

Diese Befehle laufen auf dem **Proxmox-Host**, nicht innerhalb eines Containers:

```
# Befehl in einem Container ausführen, ohne dich einzuloggen
pct exec <ID> -- <befehl>

# Beispiel: Docker-Container innerhalb von LXC 105 auflisten
pct exec 105 -- docker ps

# Interaktiv in einen Container einsteigen (wie SSH, aber ohne Passwort)
pct enter <ID>
# ... Befehle ausführen ...
exit          # zurück zum Proxmox-Host

# Container-Status prüfen (läuft er?)
pct status <ID>

# Container-Konfiguration anzeigen (Netzwerk, Ressourcen)
pct config <ID>
```

3. Installierte Software & Versionen finden

Diese Befehle zeigen dir, **was** auf einem System installiert ist und in **welcher Version** — unabhängig davon, ob du dich noch erinnerst, wie es installiert wurde.

```
# Docker-Container auflisten (nur laufende)
pct exec <ID> -- docker ps

# Auch gestoppte Container mit anzeigen
pct exec <ID> -- docker ps -a
# Komplette Konfiguration eines Docker-Containers ansehen
```

```
pct exec <ID> -- docker inspect <containername>

# Nur die Volumes (Speicherorte) eines Containers anzeigen
pct exec <ID> -- docker inspect <containername> --format '{{json .Mounts}}'

# Pfad zur docker-compose.yml eines Containers finden
pct exec <ID> -- docker inspect <containername> --format \
    '{{index .Config.Labels "com.docker.compose.project.config_files"}}'

# Versionsnummern von direkt installierter Software prüfen
pct exec <ID> -- apache2 -v
pct exec <ID> -- php -v
pct exec <ID> -- mysql --version

# systemd-Service-Status prüfen (bei direkt installierten Diensten)
pct exec <ID> -- systemctl status <servicename>
```

4. Konfigurationsdateien & Installationsordner finden

```
# Eine Datei mit bestimmtem Namen im ganzen System suchen
pct exec <ID> -- find / -maxdepth 3 -iname "<dateiname>" 2>/dev/null

# Beispiel: Nextcloud-Installationsordner finden
pct exec <ID> -- find / -maxdepth 3 -iname "nextcloud" -type d 2>/dev/null

# Inhalt einer gefundenen Datei anzeigen
pct exec <ID> -- cat /pfad/zur/datei

# Ordnerinhalt mit Details auflisten (Rechte, Größe, Datum)
pct exec <ID> -- ls -la /pfad/zum/ordner
```

Wichtig: 2>/dev/null am Ende blendet Fehlermeldungen aus (z.B. "Permission denied" bei Systemordnern, die man nicht durchsuchen darf) — so bleibt nur das eigentliche Suchergebnis übrig.

5. Dateien vom Server auf den eigenen PC holen

Diese Befehle laufen in **Git Bash auf dem Windows-PC**, nicht auf dem Server:

```
# Einzelne Datei kopieren (Punkt am Ende = in aktuellen Ordner)
scp root@<SERVER-IP>:/pfad/zur/datei.yml .

# Kompletten Ordner kopieren
scp -r root@<SERVER-IP>:/pfad/zum/ordner .

# Ordner-Inhalt statt neuen Unterordner kopieren
# (Punkt am Ende ist entscheidend - sonst entsteht ordner/ordner)
scp -r root@<SERVER-IP>:/pfad/zum/ordner .
```

6. Terminal-Grundlagen (Bash)

pwd	# zeigt den aktuellen Ordner ("wo bin ich?")
ls	# listet Dateien/Ordner im aktuellen Ordner
ls -la	# wie ls, aber mit Details + versteckten Dateien (.env etc.)
cd <ordner>	# in einen Ordner wechseln
cd ..	# eine Ebene nach oben
cd /c/IT-Projekt/...	# absoluter Pfad (funktioniert von überall)
mkdir <name>	# neuen Ordner anlegen
mv <quelle> <ziel>	# Datei/Ordner verschieben oder umbenennen
cat <datei>	# Dateiinhalt anzeigen
nano <datei>	# Datei im Texteditor öffnen (Speichern: Strg+O, Enter, Strg+X)

7. Git: Änderungen zu GitHub hochladen

Der Standard-Ablauf, um lokale Änderungen ins Repository zu bringen:

```
# 1. Prüfen, was sich geändert hat (WICHTIG vor jedem Commit!)
git status

# 2. Alle Änderungen zum Hochladen vormerken
git add .

# 3. Änderungen mit einer Beschreibung im Verlauf speichern
git commit -m "Kurze Beschreibung, was sich geändert hat"

# 4. Tatsächlich zu GitHub hochladen
git push
```

Zusätzliche, oft nützliche Git-Befehle

```
git log --oneline -5      # letzte 5 Commits kurz anzeigen
git status --ignored      # zeigt auch, was durch .gitignore ausgeschlossen wird
cat .gitignore            # aktuelle Ausschluss-Regeln anzeigen
```

Sicherheits-Check vor jedem Push: Immer erst `git status` ausführen und prüfen, dass keine Dateien mit echten Passwörtern, API-Keys oder Tokens in der Liste auftauchen (z.B. `.env`, `proxmox.yml`, `copy-key.yml`). Diese gehören in die `.gitignore`, niemals in einen Commit.

8. Typischer Gesamt-Ablauf für einen neuen Service

```
# 1. Herausfinden, wie der Service läuft
pct exec <ID> -- docker ps

# 2. Pfad zur Konfigurationsdatei finden
pct exec <ID> -- docker inspect <name> --format \
'{{index .Config.Labels "com.docker.compose.project.config_files"}}'

# 3. Inhalt sicherheitsprüfen, BEVOR man sie herunterlädt
pct exec <ID> -- cat /pfad/zur/docker-compose.yml

# 4. Datei auf den PC holen (in Git Bash, im richtigen Zielordner)
cd /c/IT-Projekt/Github/homelab/<service-ordner>
scp root@<IP>:/pfad/zur/docker-compose.yml .

# 5. README.md für den Service schreiben
nano README.md

# 6. Vor dem Hochladen: sicherstellen, dass keine Secrets drin sind
cd /c/IT-Projekt/Github/homelab
git status

# 7. Hochladen
git add .
git commit -m "Beschreibung"
git push
```